

MeetUp!

Service Specification Document

v1.0 — Project Initiation

An AI-assisted collaboration system with persistent meeting context and dynamic group formation

1. System Overview

MeetUp! is a collaboration platform built on Spring Boot microservices. Unlike traditional meeting tools, MeetUp! maintains persistent context across sessions — tracking decisions, tasks, and group relationships so that every meeting builds on what came before.

The system is organised around five core services plus an API gateway and authentication layer, each with its own database chosen to fit its data model.

1.1 Core Principles

- **Persistent context:** meetings are not isolated. Every discussion, decision, and task is stored and surfaced in future sessions.
- **Dynamic groups (Tweening):** teams are not static. Groups form organically around tasks, evolve as work changes, and dissolve when done.
- **Event-driven:** all services communicate via a message bus. No service calls another directly.
- **Intelligence from relationships:** the graph of who worked with whom, on what, and when is the primary source of system intelligence — not AI predictions.
- **AI as a tool:** AI features are bounded, purposeful additions on top of a relational/graph foundation.

1.2 User Roles

Role	Description	Permissions
Meeting Owner	Creates and controls a meeting session	Create meetings, spawn sub-meetings, create tasks, manage participants, end meeting
Group Lead	Leads a tween group assigned to a task	Manage group members, assign sub-tasks, moderate group chat, transfer leadership
Worker	Member of one or more tween groups	Join/leave groups, update task status, participate in meetings and chat

1.3 Technology Stack

Layer	Technology
Backend framework	Spring Boot 3.x (Java 21)
Message bus	Apache Kafka (or RabbitMQ)
API gateway	Spring Cloud Gateway
Authentication	Supabase Auth (JWT + social providers + OTP)
Real-time video	Stream (or Janus SFU for WebRTC)

Real-time chat	Stream Chat API
Containerisation	Docker + Docker Compose
Frontend	React (or Next.js)

2. API Gateway

The single entry point for all client requests. Routes traffic to the appropriate microservice, enforces authentication, and applies rate limiting. No business logic lives here.

2.1 Responsibilities

- Route incoming HTTP requests to the correct downstream service based on path prefix
- Validate JWT tokens on every request before forwarding
- Reject unauthenticated requests at the gateway level — services assume requests are authenticated
- Apply rate limiting per user and per IP
- Handle CORS configuration centrally
- Forward user identity headers (user ID, role) to downstream services
- Aggregate health checks from all services

2.2 Routing Table

Path prefix	Downstream service
/api/auth/**	Auth Service
/api/meetings/**	Meeting Service
/api/tasks/**	Task Service
/api/groups/**	Tweening Service
/api/context/**	Context Service
/api/ai/**	AI/NLP Service

2.3 Technology

- **Spring Cloud Gateway:** reactive routing, filter chains
- **JWT validation:** public key from Supabase, validated on every request
- **Redis:** rate limit counters, token blacklist

Service 2 — Auth Service

3. Auth Service

Delegates all user management and authentication to Supabase. The Auth Service acts as a thin adapter — it communicates with Supabase and translates responses into the JWT format the rest of the system understands. No passwords are stored in the application database.

3.1 Responsibilities

- Provide sign-up and sign-in endpoints backed by Supabase Auth
- Support social login providers: Google, GitHub (configured in Supabase dashboard)
- Support OTP-based login via email and SMS (delegated to Supabase)
- Issue and refresh JWT access tokens
- Expose a /validate endpoint for the API Gateway to verify tokens
- Manage user profile creation on first login — write user record to the shared user store
- Handle logout and token invalidation
- Publish user.registered event on new account creation

3.2 Supabase Integration

Supabase handles the following directly — the Auth Service does not re-implement these:

- Password hashing and storage
- OAuth 2.0 flows with social providers
- OTP generation and email/SMS delivery
- Session management and refresh token rotation

3.3 Events Published

Event	Trigger	Consumers
user.registered	New account created	Tweening Service (create Person node in Neo4j)
user.updated	Profile changed	Context Service, Dashboard

3.4 Database

- Supabase PostgreSQL — user records, OAuth metadata (managed by Supabase)
- Redis — token blacklist for invalidated JWTs

Service 3 — Meeting Service

4. Meeting Service

Manages the full lifecycle of meeting sessions. There are two types of meeting: short-form (lightweight, no persistent context) and long-form (full context tracking, links to tasks and groups). Sub-meetings can be spawned from a main meeting as private breakout rooms.

4.1 Meeting Types

- **Short meeting:** lightweight, ephemeral. No transcript, no context stored. Used for quick syncs.
- **Long meeting:** full context tracking. Transcript captured, decisions and tasks linked, context persisted to Context Service. This is the primary meeting type in MeetUp!.
- **Sub-meeting (spawned):** a private breakout room created from within a main meeting, typically scoped to a tween group. Inherits context from the parent meeting.

4.2 Responsibilities

- Create, configure, and destroy meeting rooms
- Manage participant joining and leaving (presence tracking)
- Spawn sub-meetings from an active main meeting
- Handle real-time video via Stream (or Janus SFU)
- Handle real-time group chat via Stream Chat, scoped per meeting and per group
- Enforce meeting controls: mute, lock room, remove participant
- Emit events on all significant state changes (started, joined, ended, sub-meeting spawned)
- Write transcript chunks to the message bus in real time for the Context Service to consume
- Link meeting records to tasks and tween groups

4.3 Stream Integration

Stream is used for both video calls and chat. The Meeting Service creates Stream sessions and chat channels and returns tokens to the client. The client connects to Stream directly — media does not pass through the Meeting Service.

- One Stream video call per meeting room
- One Stream chat channel per meeting (main)
- Additional private Stream chat channels per tween group within the meeting

4.4 Events Published

Event	Trigger	Consumers
-------	---------	-----------

meeting.started	Room created and host joins	Context Service, Tweening Service
meeting.ended	Host ends the meeting	Context Service, AI Service, Tweening Service
participant.joined	User joins room	Context Service, Tweening Service
participant.left	User leaves room	Context Service
submeeting.spawned	Breakout room created	Context Service, Tweening Service
transcript.chunk	STT segment ready	Context Service

4.5 Database

- **PostgreSQL:** meeting records, participant logs, room configuration, sub-meeting relationships

Note: All notifications triggered by meeting events must be published via the message bus, not sent directly.

Service 4 — Context Service

5. Context Service

The memory layer of MeetUp!. Every long-form meeting produces a context document — a structured record of what was discussed, what was decided, and what tasks were created. This document is queried before subsequent meetings to generate a briefing for participants.

5.1 Responsibilities

Write side — ingestion

- Consume transcript.chunk events and assemble rolling transcripts per meeting
- Perform speaker labelling using participant IDs from the session
- Deduplicate overlapping STT segments
- Consume task.created, task.assigned, task.completed events and log them as actions within the meeting context
- Consume group.formed and group.dissolved events and link them to the meeting record
- On meeting.ended: finalise the context document and mark it complete
- Cache frequently accessed briefings in Redis to avoid repeated MongoDB queries

5.2 Write side — enrichment

- Tag discussions by topic using keyword extraction (TF-IDF, no LLM required)
- Identify decision moments using pattern detection on the transcript (phrases: 'we agreed', 'let's go with', 'decided to')
- Link task references from the Task Service into the context document
- Receive and store AI-generated summary from the AI/NLP Service post-meeting

5.3 Read side — briefing builder

- Expose GET /context/briefing?meetingId=X endpoint
- Query MongoDB for prior sessions shared by the same participants or tween group
- Surface: summary of last session, decisions made, tasks still open
- Return structured JSON consumed by the client before a meeting starts

5.4 Read side — decision log

- Maintain an immutable append-only log of decisions per project/group
- Each decision record: text, attributed speaker, meeting ID, timestamp
- Expose GET /context/decisions?groupId=X for dashboard and briefing use

5.5 Read side — search

- Full-text search across all context documents for a user's meetings
- Filter by topic tag, participant, date range, group
- Powered by MongoDB text indexes

5.6 MongoDB Document Shape

One document per long-form meeting session:

Field	Type	Description
meeting_id	UUID	Links to Meeting Service record
participant_ids	UUID[]	All participants in the session
tween_group_ids	UUID[]	Groups active during this meeting
transcript	Object[]	Chunks: speaker, text, timestamp
decisions	Object[]	Text, speaker, timestamp — immutable
tasks_ref	UUID[]	Task IDs created/discussed in this meeting
topics	String[]	Extracted topic tags
summary	String	AI-generated summary (populated post-meeting)
status	Enum	live complete

5.7 Database

- **MongoDB:** primary context documents store
- **Redis:** briefing cache, hot context for live meetings

Service 5 — Tweening Service

6. Tweening Service

Manages dynamic group (tween) creation, membership, and lifecycle. Groups are always task-anchored — they exist to do a specific piece of work, and they dissolve when that work is done. The full history of group relationships is stored in Neo4j as a property graph.

6.1 Group Lifecycle States

State	Description	Trigger
Latent	Task exists, no group yet formed	task.created event
Forming	Group created, members being invited	Meeting owner or Group Lead creates group
Active	At least two members collaborating	Second member accepts invitation
Evolving	Group splitting, merging, or membership changing	Task split or member transfer
Dissolved	Task complete, group archived	task.completed event

6.2 Responsibilities

Group management

- Handle group creation — always linked to a task and a meeting
- Handle member joining — append MEMBER_OF relationship in Neo4j with joined_at timestamp
- Handle member exit — end MEMBER_OF relationship, record left_at
- Handle group transfer — change of Group Lead, update LEADS relationship
- Handle group dissolve — mark dissolved, archive chat and resources
- Map dynamic roles within a group — roles (Lead, Worker) can change over time, all changes recorded as graph events
- Support manual group split — one group becomes two, both linked as siblings in the graph
- Support manual group merge — two groups become one, relationships consolidated

6.3 Membership resolver

- When a task is created, suggest members using Neo4j graph traversal
- Query: who has collaborated with the task creator before, weighted by recency
- Query: who has worked on tasks with similar topic tags
- Return ranked suggestions — no LLM involved, pure graph traversal

6.4 Neo4j Graph Model

Node type	Properties
Person	id, name, email, role (system-wide)
Group	id, name, state, task_id, created_at, dissolved_at
Task	id, title, status, priority
Meeting	id, title, type, started_at

Relationship	From → To	Properties
MEMBER_OF	Person → Group	joined_at, left_at, role_in_group
LEADS	Person → Group	from, to
WORKS_ON	Group → Task	assigned_at
FORMED_IN	Group → Meeting	spawned_at
COLLABORATED_WITH	Person → Person	weight, last_collaboration

Note: COLLABORATED_WITH is a derived relationship. It is written when a group dissolves, weighted by collaboration duration. This is what powers the membership resolver.

6.5 Events Published

Event	Trigger	Consumers
group.formed	Group created	Meeting Service, Context Service, Task Service
group.evolved	Split, merge, or role change	Context Service, Dashboard
group.dissolved	Task completed	Context Service, Task Service, AI Service
member.joined	User accepts invitation	Context Service, Meeting Service
member.left	User exits group	Context Service

Service 6 — Task Service

7. Task Service

Manages the lifecycle of tasks. Tasks are the structural anchor of MeetUp! — they drive group formation, link meeting discussions to deliverables, and provide the signal that tells the Tweening Service when groups should dissolve.

7.1 Task State Machine

State	Description	Transition trigger
Backlog	Task created, not yet picked up	task.created
In progress	Group active on this task	group.formed
Blocked	Dependency not yet resolved	dependency task still open
Review	Work done, awaiting sign-off	Group Lead marks ready
Done	Signed off and closed	Meeting Owner or Group Lead approves

7.2 Responsibilities

- Create tasks — by Meeting Owner during or after a meeting, or as AI-extracted drafts awaiting confirmation
- Assign tasks to participants — triggers group formation in Tweening Service
- Send invitations to each assigned person to join the task group (delegated to Meeting Service for sub-meeting creation)
- On acceptance: participant joins the tween group, sub-meeting is activated with chat and video
- Handle task modification — title, description, priority, due date, assignees
- Manage subtasks — parent/child task relationships
- Manage task dependencies — blocked state enforced when dependencies are open
- Handle task comments — append-only log per task
- Expose task history — full audit trail of state transitions and assignments

7.3 Events Published

Event	Trigger	Consumers
task.created	New task saved	Tweening Service (suggest group), Context Service, AI Service
task.assigned	Assignee added	Tweening Service (form group), Meeting Service (invite)

task.updated	Any field change	Context Service, Dashboard
task.completed	State → Done	Tweening Service (dissolve group), Context Service, AI Service

7.4 Database

- **PostgreSQL:** tasks, subtasks, dependencies, assignments, comments

Key indexes: (state, assignee_id) for dashboard queries, (created_in_meeting_id) for context linking, (tween_group_id) for group task board.

8. AI / NLP Service

A bounded, purposeful service. AI features are tools applied to specific problems, not the foundation of the system. The service exposes four capabilities, each triggered by a specific event. All LLM calls are wrapped behind a pluggable provider interface so the implementation can be swapped without touching feature code.

8.1 Features

Meeting summariser

- Triggered by: meeting.ended
- Input: full transcript from Context Service
- Process: structured LLM prompt requesting summary, decisions list, and open questions in JSON format
- Output: writes summary JSON back to Context Service via REST call
- Provider: OpenAI GPT-4o (production) / Ollama Llama3 (local dev via Spring AI)

Task extractor

- Triggered by: meeting.ended
- Input: transcript from Context Service
- Process tier 1: rule-based pattern matching for action-signalling phrases ('I will', 'can you take', 'we need to')
- Process tier 2: LLM pass on unmatched segments to catch subtler commitments
- Output: draft tasks published to Task Service as status=draft — require user confirmation before becoming real tasks

Assignee suggester

- Triggered by: task.created
- Input: task title, topic tags, creator ID
- Process: Cypher query against Neo4j COLLABORATED_WITH graph, ranked by weight and recency
- Output: ordered list of suggested assignees returned to the Dashboard — no LLM involved

Topic tagger

- Triggered by: transcript.chunk (runs live during meeting)
- Input: transcript chunk text
- Process: TF-IDF keyword extraction against a domain vocabulary — no LLM
- Output: topic tags appended to the live context document in MongoDB

8.2 Provider layer

All LLM calls go through a `LanguageModelClient` interface. Spring AI provides the abstraction. Configuration switches between providers via `application.yml`.

Environment	Provider	Notes
Local development	Ollama (Llama 3)	Free, runs on-device, no API costs
Demo / production	OpenAI GPT-4o	Swap via config, no code changes

8.3 Events Published

Event	Trigger	Consumers
ai.summary.ready	Summary generated	Context Service (stores result)
ai.tasks.extracted	Draft tasks identified	Task Service, Dashboard (for confirmation)

9. Shared Infrastructure

9.1 Message Bus

All inter-service communication happens via Kafka (or RabbitMQ). Services publish events and subscribe to topics. No service calls another's REST API except: AI Service → Context Service (to write summary), and API Gateway → Auth Service (to validate tokens).

Note: All notifications must be published via the message bus. No direct service-to-service notification calls.

9.2 Notifications

- All in-app notifications (task assigned, group invite, meeting starting, summary ready) are delivered via the message bus
- A dedicated Notification Consumer subscribes to relevant events and dispatches push notifications or in-app alerts
- Email notifications are sent via Supabase or a transactional email provider (Resend/Postmark)

9.3 Docker Compose

A single docker-compose.yml bootstraps the full local environment:

- All five services
- PostgreSQL (Meeting Service, Task Service, Auth metadata)
- MongoDB (Context Service)
- Neo4j (Tweening Service)
- Redis (API Gateway rate limiting, Context Service cache)
- Kafka + Zookeeper (or RabbitMQ)

9.4 Shared Events Module

A shared Maven/Gradle module (meetup-events) defines all event DTOs used across services. All services import this module. Event schemas are locked after week two of development — changes require team sign-off.